

OPERATING SYSTEMS (UE24CS242B)

Unit 2 – IPC, Threads and Concurrency

Lecture 1 - Shared Memory and Message Passing

Interprocess Communication (IPC)

Processes within a system may be either:

- Independent processes
- Cooperating processes

A cooperating process can affect or be affected by the execution of another process, typically by sharing data.

To enable cooperation, processes require Interprocess Communication (IPC) mechanisms.

Reasons for Cooperating Processes

Cooperating processes are useful for several reasons:

1. Information Sharing
 - Multiple users or processes need access to shared information
 - Example: Shared database
2. Computation Speed-up
 - Tasks are divided into sub-tasks
 - Sub-tasks execute in parallel on multiple processes
3. Modularity
 - System is divided into separate modules
 - Each module runs as a separate process
4. Convenience
 - Allows multiple activities to run concurrently
 - Example: Editing, printing, and compiling at the same time

Cooperating vs Independent Processes

Independent Process

- Cannot affect or be affected by other processes
- Does not share data with other processes

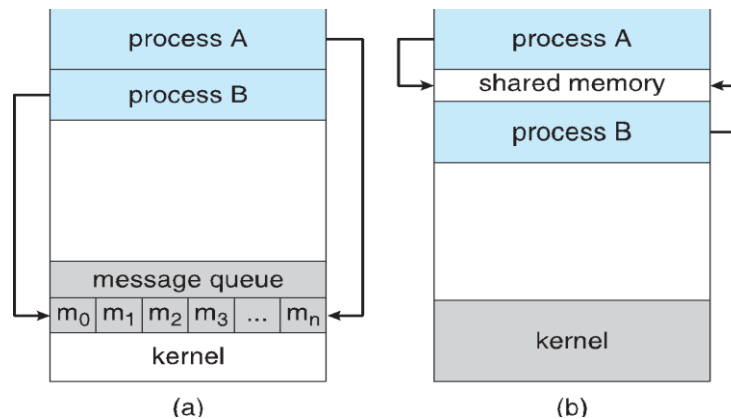
Cooperating Process

- Can affect or be affected by other processes
- Requires IPC mechanisms

IPC Models

There are two fundamental models of Interprocess Communication:

1. Shared Memory
2. Message Passing



Producer–Consumer Problem

The Producer–Consumer problem is a classical example of cooperating processes.

- Producer process
 - Produces information (data items)
- Consumer process
 - Consumes the information produced

Types of Buffers in Producer–Consumer

Unbounded Buffer

- No practical limit on buffer size
- Producer can always produce items
- Consumer may have to wait if buffer is empty

Bounded Buffer

- Fixed buffer size
- Producer must wait if buffer is full
- Consumer must wait if buffer is empty

Bounded-Buffer – Shared Data

Shared buffer is implemented as a circular array.

Shared Data Structures

```
#define BUFFER_SIZE 10
```

```
typedef struct {
```

```
...
```

```
} item;
```

```
item buffer[BUFFER_SIZE];
```

```
int in = 0;
```

```
int out = 0;
```

Buffer Conditions

- Buffer empty $\rightarrow in == out$
- Buffer full $\rightarrow ((in + 1) \% BUFFER_SIZE) == out$

Pointer Meaning

- $in \rightarrow$ points to the next free position
- $out \rightarrow$ points to the first full position

Note: This solution can use only $BUFFER_SIZE - 1$ elements.

Bounded-Buffer – Producer Code

```
item next_produced;
```

```
while (true) {
```

```
    /* produce an item in next_produced */
```

```
    while (((in + 1) \% BUFFER\_SIZE) == out)
```

```
        ; /* do nothing */
```

```
    buffer[in] = next_produced;
```

```
    in = (in + 1) \% BUFFER\_SIZE;
```

```
}
```

- Producer waits if buffer is full
- Busy waiting is used (inefficient but conceptually simple)

Bounded-Buffer – Consumer Code

```
item next_consumed;
```

```
while (true) {
```

```
    while (in == out)
```

```
        ; /* do nothing */  
  
        next_consumed = buffer[out];  
        out = (out + 1) % BUFFER_SIZE;  
        /* consume the item */  
    }
```

- Consumer waits if buffer is empty
- Busy waiting is again used

Interprocess Communication – Shared Memory

Shared Memory Concept

- A region of memory is shared among cooperating processes
- Communication is controlled by user processes, not by the OS
- OS provides only the mechanism, not the synchronization

Shared Memory – Key Points

- Two or more processes share the same memory region
- Fastest IPC mechanism
 - No data copying between processes
- Main challenge:
 - Synchronization

Processes must coordinate access to avoid data inconsistency.

Synchronization in Shared Memory

- Processes must not access shared memory simultaneously in a conflicting way
- Common synchronization tools:
 - Semaphores
 - Mutex locks
 - Record locking

Interprocess Communication – Message Passing

Message passing is another mechanism for IPC where:

- Processes communicate without shared variables

- OS provides send and receive operations

IPC Operations

- send(message)
- receive(message)

Message size may be:

- Fixed
- Variable

Message Passing – Basic Requirements

For processes P and Q to communicate:

1. Establish a communication link
2. Exchange messages using send/receive

Implementation Issues in Message Passing

Important design questions include:

- How are communication links established?
- Can a link be shared by more than two processes?
- How many links can exist between a pair of processes?
- What is the capacity of the link?
- Is message size fixed or variable?
- Is communication:
 - Unidirectional?
 - Bi-directional?

Implementation of Communication Links

Physical Implementation

- Shared memory
- Hardware bus
- Network

Logical Implementation

- Direct or indirect communication
- Synchronous or asynchronous communication
- Automatic or explicit buffering

Direct Communication

Definition

Processes explicitly name each other.

Operations

- $\text{send}(P, \text{message}) \rightarrow$ send message to process P
- $\text{receive}(Q, \text{message}) \rightarrow$ receive message from process Q

Properties

- Links established automatically
- Exactly one link per process pair
- Usually bi-directional

Indirect Communication

Definition

Messages are sent and received via mailboxes (ports).

- Each mailbox has a unique ID
- Processes communicate only if they share a mailbox

Indirect Communication – Properties

- Link established only if processes share a mailbox
- A link may be associated with multiple processes
- A process pair may share multiple links
- Communication can be:
 - Unidirectional
 - Bi-directional

Mailbox Operations

Operations supported:

- Create a mailbox
- Send message to mailbox
- Receive message from mailbox
- Destroy mailbox

Primitives

- $\text{send}(A, \text{message})$
- $\text{receive}(A, \text{message})$

Mailbox Sharing Problem

Scenario:

- Processes P_1, P_2, P_3 share mailbox A

- P₁ sends a message
- P₂ and P₃ are ready to receive

Possible Solutions

1. Allow link to be associated with at most two processes
2. Allow only one process to execute receive at a time
3. System selects receiver arbitrarily and notifies sender

Message Passing – Synchronization

Message passing can be:

Blocking (Synchronous)

- Blocking send → sender waits until message is received
- Blocking receive → receiver waits until message arrives

Non-Blocking (Asynchronous)

- Non-blocking send → sender continues immediately
- Non-blocking receive → receiver gets message or null

Rendezvous

- Both send and receive are blocking
- Sender and receiver meet at the same time

Producer–Consumer using Message Passing

Producer

```
message next_produced;
```

```
while (true) {  
    /* produce item */  
    send(next_produced);  
}
```

Consumer

```
message next_consumed;
```

```
while (true) {  
    receive(next_consumed);  
    /* consume item */  
}
```

Note: Message passing eliminates shared memory synchronization issues

Buffering in Message Passing

Messages are stored in queues attached to the communication link.

Buffering Types

1. Zero Capacity
 - No messages queued
 - Sender must wait for receiver
 - Also called no buffering (rendezvous)
2. Bounded Capacity
 - Queue size = n messages
 - Sender waits if queue is full
3. Unbounded Capacity
 - Infinite queue size
 - Sender never waits